

Programação Orientada a Objetos

Professor MSc. Ronilson Pereira
ronilsonengenharia@gmail.com





Começando a resolver um segundo problema exemplo

Prof. MSc. Ronilson Pereira

Outro exemplo

Fazer um programa para ler os dados de um produto em estoque (nome, preço e quantidade no estoque). Em seguida:

- Mostrar os dados do produto (nome, preço, quantidade no estoque, valor total no estoque)
- Realizar uma entrada no estoque e mostrar novamente os dados do produto
- Realizar uma saída no estoque e mostrar novamente os dados do produto

Para resolver este problema, você deve criar uma CLASSE conforme projeto ao lado:

(veja exemplo na próxima página)

Product
- Name : string - Price : double - Quantity : int
+ TotalValueInStock() : double + AddProducts(quantity : int) : void + RemoveProducts(quantity : int) : void

Exemplo

Insira os dados do produto:

Nome: **TV**

Preço: **900.00**

Quantidade em estoque: **10**

Dados do Produto: TV, R\$ 900.00, 10 unidades, Total: \$ 9000.00

Insira a quantidade de produtos a serem adicionados ao estoque: **5**

Dados Atualizados: TV, \$ 900.00, 15 unidades, Total: \$ 13500.00

Insira a quantidade de produtos a serem removidos do estoque: **3**

Dados Atualizados: TV, \$ 900.00, 12 unidades, Total: \$ 10800.00

Product

- Name : string
- Price : double
- Quantity : int

- + TotalValueInStock() : double
- + AddProducts(quantity : int) : void
- + RemoveProducts(quantity : int) : void



Object e toString

Prof. MSc. Ronilson Pereira



Discussão

- Toda classe em Java é uma subclasse da classe Object
- Object possui os seguintes métodos:
 - getClass- retorna o tipo do objeto
 - equals - compara se o objeto é igual a outro
 - hashCode - retorna um código hash do objeto
 - toString - converte o objeto para string



Membros estáticos – Parte I

Prof. MSc. Ronilson Pereira



Product
- Name : string - Price : double - Quantity : int
+ TotalValueInStock() : double + AddProducts(quantity : int) : void + RemoveProducts(quantity : int) : void

membros
=
atributos e métodos



Membros estáticos

- Também chamados membros de classe
 - Em oposição a membros e instância
- São membros que fazem sentido independentemente de objetos. Não precisam de objeto para serem chamados. São chamados a partir do próprio nome da classe.
- Aplicações comuns:
 - Classes utilitárias → **Math.sqrt(double)**
 - Declaração de constantes
- Uma classe que possui somente membros estáticos, pode ser uma classe estática também. Esta classe não poderá ser instanciada.



```
Triangle x, y;  
x = new Triangle();  
y = new Triangle();
```



x.area() → 6.0

y.area() → 7.5638



Problema exemplo

Fazer um programa para ler um valor numérico qualquer, e daí mostrar quanto seria o valor de uma circunferência e do volume de uma esfera para um raio daquele valor. Informar também o valor de PI com duas casas decimais.

Exemplo:

```
Enter radius: 3.0  
Circumference: 18.85  
Volume: 113.10  
PI value: 3.14
```



Checklist

- Versão 1: métodos na própria classe do programa
 - Nota: dentro de um método estático você não pode chamar membros de instância da mesma classe.
- Versão 2: classe Calculator com membros de instância
- Versão 3: classe Calculator com método estático

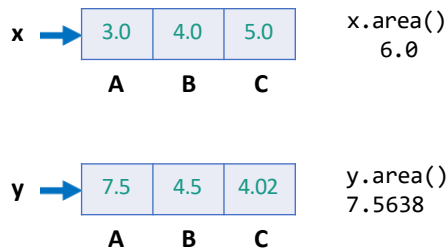


Membros estáticos – Parte II

Prof. MSc. Ronilson Pereira

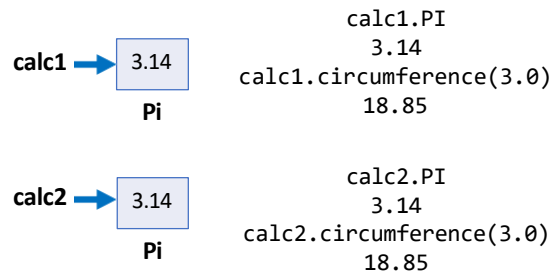
Discussão

- No problema dos triângulos, cada triângulo possui sua área.
- Area() é uma operação concernente ao objeto: cada triângulo possui sua área.



- Já no caso da calculadora, os valores dos cálculos não mudam para calculadoras diferentes, ou seja, são cálculos estáticos. O valor de Pi também é estático.

```
Calculator calc1 = new Calculator();  
Calculator calc2 = new Calculator();
```





```
package util;

public class Calculator {

    public static final double PI = 3.14159;

    public static double circumference(double radius) {
        return 2.0 * PI * radius;
    }

    public static double volume(double radius) {
        return 4.0 * PI * radius * radius * radius / 3.0;
    }
}
```

```
System.out.print("Enter radius: ");
double radius = sc.nextDouble();

double c = Calculator.circumference(radius);

double v = Calculator.volume(radius);

System.out.printf("Circumference: %.2f\n", c);
System.out.printf("Volume: %.2f\n", v);
System.out.printf("PI value: %.2f\n", Calculator.PI);
```

VERSÃO 3



POO - Java

Construtores, palavra this,
sobrecarga, encapsulamento

Prof. MSc. Ronilson Pereira



Construtores

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira



Construtor

- É uma operação especial da classe, que executa no momento da instanciação do objeto
- Usos comuns:
 - Iniciar valores dos atributos
 - Permitir ou obrigar que o objeto receba dados / dependências no momento de sua instanciação (injeção de dependência)
- Se um construtor customizado não for especificado, a classe disponibiliza o construtor padrão:
`Product p = new Product();`
- É possível especificar mais de um construtor na mesma classe (sobrecarga)



Problema Exemplo

Insira os dados do produto:

Nome: **TV**

Preço: **900.00**

Quantidade em estoque: **10**

Dados do Produto: TV, R\$ 900.00, 10 unidades, Total: \$ 9000.00

Insira a quantidade de produtos a serem adicionados ao estoque: **5**

Dados Atualizados: TV, \$ 900.00, 15 unidades, Total: \$ 13500.00

Insira a quantidade de produtos a serem removidos do estoque: **3**

Dados Atualizados: TV, \$ 900.00, 12 unidades, Total: \$ 10800.00

Product
- Name : string - Price : double - Quantity : int
+ TotalValueInStock() : double + AddProducts(quantity : int) : void + RemoveProducts(quantity : int) : void

Proposta de melhoria

Quando executamos o comando abaixo, instanciamos um produto "**product**" com seus atributos "vazios":

```
product = new Product();
```

Memória:



Entretanto, faz sentido um produto que não tem nome? Faz sentido um produto que não tem preço?

Com o intuito de evitar a existência de produtos sem nome e sem preço, é possível fazer com que seja "obrigatória" a iniciação desses valores?



```
public class Product {  
  
    public String name;  
    public double price;  
    public int quantity;  
  
    public Product(String name, double price, int quantity) {  
        this.name = name;  
        this.price = price;  
        this.quantity = quantity;  
    }  
    (...)
```

```
System.out.println("Enter product data: ");  
System.out.print("Name: ");  
String name = sc.nextLine();  
System.out.print("Price: ");  
double price = sc.nextDouble();  
System.out.print("Quantity in stock: ");  
int quantity = sc.nextInt();  
Product product = new Product(name, price, quantity);
```



Palavra this

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira



Palavra this

- É uma referência para o próprio objeto
- Usos comuns:
 - Diferenciar atributos de variáveis locais
 - Passar o próprio objeto como argumento na chamada de um método ou construtor

Diferenciar atributos de variáveis locais

```
Product product = new Product("TV", 1500.0, 0);
```

```
public Product(String name, double price, int quantity) {  
    this.name = name;  
    this.price = price;  
    this.quantity = quantity;  
}
```

Memória

(escopo do construtor)

"TV"
name

1500.0
price

0
quantity

product



"TV"	1500.0	0
name	price	quantity

this.name





Passar o próprio objeto como argumento na chamada de um método ou construtor

```
public class ChessMatch {  
    (...)  
    placeNewPiece('e', 1, new King(board, Color.WHITE, this));  
    (...)  
}
```



Sobrecarga

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira



Sobrecarga

- É um recurso que uma classe possui de oferecer mais de uma operação com o mesmo nome, porém com diferentes listas de parâmetros.



Proposta de melhoria

- Vamos criar um construtor opcional, o qual recebe apenas nome e preço do produto. A quantidade em estoque deste novo produto, por padrão, deverá então ser iniciada com o valor zero.
- Nota: é possível também incluir um construtor padrão



```
public class Product {  
  
    public String name;  
    public double price;  
    public int quantity;  
  
    public Product() {  
    }  
  
    public Product(String name, double price, int quantity) {  
        this.name = name;  
        this.price = price;  
        this.quantity = quantity;  
    }  
  
    public Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
    (...)
```



Encapsulamento

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira

Encapsulamento

- É um princípio que consiste em esconder detalhes de implementação de uma classe, expondo apenas operações seguras e que mantenham os objetos em um estado consistente.
- Regra de ouro: o objeto deve sempre estar em um estado consistente, e a própria classe deve garantir isso.

Analogia:





Regra geral básica

- Um objeto **NÃO** deve expor nenhum atributo (modificador de acesso **private**)
- Os atributos devem ser acessados por meio de métodos get e set
 - Padrão JavaBeans: <https://en.wikipedia.org/wiki/JavaBeans>



Padrão para implementação de getters (obter) e setters(alterar)

```
private String nome;  
private double preco;  
  
public String getNome() {  
    return nome;  
}  
  
public void setNome(String nome) {  
    this.nome = nome;  
}  
  
public double getPreco() {  
    return preco;  
}  
  
public void setPreco(double preco) {  
    this.preco = preco;  
}
```



```
public class Product {  
  
    private String name;  
    private double price;  
    private int quantity;  
  
    public Product() {  
    }  
  
    public Product(String name, double price, int quantity) {  
        this.name = name;  
        this.price = price;  
        this.quantity = quantity;  
    }  
  
    public Product(String name, double price) {  
        this.name = name;  
        this.price = price;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public double getPrice() {  
        return price;  
    }  
  
    public void setPrice(double price) {  
        this.price = price;  
    }  
  
    public int getQuantity() {  
        return quantity;  
    }  
  
    (...)  
}
```



Gerando automaticamente construtores , getters e setters com Eclipse

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira



Comandos

- Botão direito -> Source -> Generate Constructor using Fields
- Botão direito -> Source -> Generate Getters and Setters



Modificadores de acesso

ronilsonengenharia@gmail.com

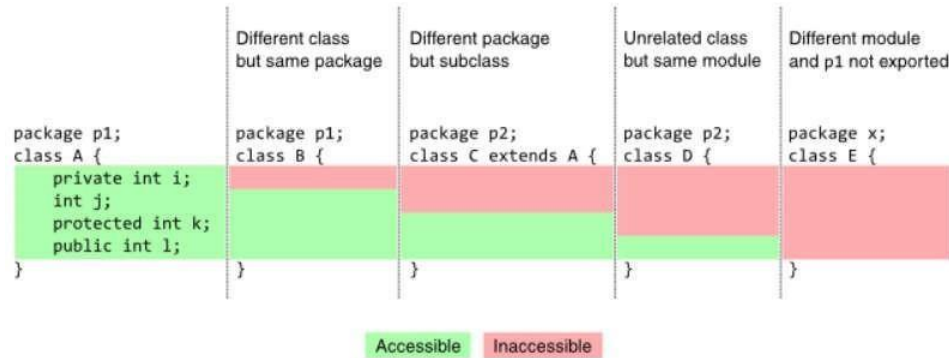
Prof. MSc. Ronilson Pereira



Modificadores de acesso

- <https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html>
- **private**: o membro só pode ser acessado na **própria classe**
- (nada): o membro só pode ser acessado nas classes do **mesmo pacote**
- **protected**: o membro só pode ser acessado no **mesmo pacote**, bem como em **subclasses de pacotes diferentes**
- **public**: o membro é acessado por todas classes (ao menos que ele resida em um módulo diferente que não exporte o pacote onde ele está)

<https://stackoverflow.com/questions/215497/in-java-difference-between-package-private-public-protected-and-private>





Exercício de Fixação

ronilsonengenharia@gmail.com

Prof. MSc. Ronilson Pereira



Em um banco, para se cadastrar uma conta bancária, é necessário informar o número da conta, o nome do titular da conta, e o valor de depósito inicial que o titular depositou ao abrir a conta. Este valor de depósito inicial, entretanto, é opcional, ou seja: se o titular não tiver dinheiro a depositar no momento de abrir sua conta, o depósito inicial não será feito e o saldo inicial da conta será, naturalmente, zero.

Importante: uma vez que uma conta bancária foi aberta, o número da conta nunca poderá ser alterado. Já o nome do titular pode ser alterado (pois uma pessoa pode mudar de nome por ocasião de casamento, por exemplo).

Por fim, o saldo da conta não pode ser alterado livremente. É preciso haver um mecanismo para proteger isso. O saldo só aumenta por meio de depósitos, e só diminui por meio de saques. Para cada saque realizado, o banco cobra uma taxa de \$ 5.00. Nota: a conta pode ficar com saldo negativo se o saldo não for suficiente para realizar o saque e/ou pagar a taxa.

Você deve fazer um programa que realize o cadastro de uma conta, dando opção para que seja ou não informado o valor de depósito inicial. Em seguida, realizar um depósito e depois um saque, sempre mostrando os dados da conta após cada operação.

(exemplos nas próximas páginas)



EXEMPLO 1

Insira o número da conta: **8532**
Insira o titular da conta: **Alex Green**
Existe um depósito inicial (s/n)? **s**
Insira o valor do depósito inicial: **500.00**

Dados da conta:
Conta 8532, Titular: Alex Green, Balanço: R\$ 500.00

Insira um valor de depósito: **200.00**
Dados da conta atualizados:
Conta 8532, Titular: Alex Green, Balanço: \$ 700.00

Insira um valor de saque: **300.00**
Dados da conta atualizados:
Conta 8532, Titular: Alex Green, Balanço: \$ 395.00

Obrigado!

Dúvidas?

Ronilson Pereira

ronilsonengenharia@gmail.com

